

Feature Selection, Model Evaluation and Boruta

A beginner-friendly record of the code and concepts discussed

1. Overall Workflow

The practical uses the Scikit-learn Wine Recognition dataset and a Gradient Boosting Classifier to compare feature-selection methods. The workflow is: load the data, separate features from the target, split into training and testing sets, train a baseline model, apply different feature-selection techniques, evaluate each version using weighted F1-score, and compare the results in a bar chart.

2. Loading the Wine Dataset

```
from sklearn.datasets import load_wine
wine_data = load_wine()
wine_df = pd.DataFrame(wine_data.data, columns=wine_data.feature_names)
wine_df['target'] = wine_data.target
```

Explanation: `load_wine()` loads Scikit-learn's built-in Wine Recognition dataset. It contains 178 observations, 13 input features, and three target classes. The `DataFrame` stores the feature values and then adds the target as a separate column.

3. Separating X and y and Splitting the Data

```
from sklearn.model_selection import train_test_split

X = wine_df.drop(['target'], axis=1)
y = wine_df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, shuffle=True,
    stratify=y, random_state=42
)
```

Explanation: `X` contains input features and `y` contains the target. A test size of 0.3 keeps 30% for testing and 70% for training. Shuffling randomizes the observations, stratification preserves class proportions, and `random_state` makes the split reproducible.

4. Baseline Gradient Boosting Model

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import f1_score

gbc = GradientBoostingClassifier(max_depth=5, random_state=42)
gbc.fit(X_train, y_train)
preds = gbc.predict(X_test)
f1_score_all = round(
    f1_score(y_test, preds, average='weighted'), 3
)
```

Explanation: `fit()` trains the Gradient Boosting Classifier. `predict()` produces predictions. Weighted F1-score evaluates the predictions while accounting for the sizes of the different classes. This baseline becomes the reference for the feature-selection experiments.

5. Independent Copies

```
X_train_v1, X_test_v1, y_train_v1, y_test_v1 = (
    X_train.copy(), X_test.copy(),
```

```
y_train.copy(), y_test.copy()
)
```

copy() creates independent copies so that different feature-selection experiments can be performed without accidentally changing the original data.

6. Variance-Based Selection

```
X_train_v1.var(axis=0)
```

This calculates variance for every feature column. axis=0 means the calculation is performed down each column.

```
sel_X_train_v1 = X_train_v1.drop(['ash', 'magnesium'], axis=1)
sel_X_test_v1 = X_test_v1.drop(['ash', 'magnesium'], axis=1)

gbc.fit(sel_X_train_v1, y_train_v1)
var_preds = gbc.predict(sel_X_test_v1)
f1_score_var = round(
    f1_score(y_test_v1, var_preds, average='weighted'), 3
)
```

This experiment removes the selected low-variance features, retrains the model, predicts on the test set, and calculates weighted F1-score. Low variance does not automatically mean a feature is useless; it is simply the basis of this selection experiment.

7. SelectKBest with Mutual Information

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import mutual_info_classif

f1_score_list = []

for k in range(1, 14):
    selector = SelectKBest(mutual_info_classif, k=k)
    selector.fit(X_train_v2, y_train_v2)

    sel_X_train_v2 = selector.transform(X_train_v2)
    sel_X_test_v2 = selector.transform(X_test_v2)

    gbc.fit(sel_X_train_v2, y_train_v2)
    kbest_preds = gbc.predict(sel_X_test_v2)

    f1_score_kbest = round(
        f1_score(y_test_v2, kbest_preds, average='weighted'), 3
    )
    f1_score_list.append(f1_score_kbest)
```

Explanation: SelectKBest ranks features using Mutual Information and keeps the top k. The loop tests every value from 1 to 13. The selector is fitted on training data, the same selection is applied to test data, the model is retrained, and the resulting F1-score is stored. Mutual Information measures how informative each feature is about the target.

8. Finding Selected Feature Names with SelectKBest

```
selector = SelectKBest(mutual_info_classif, k=3)
selector.fit(X_train_v2, y_train_v2)

selected_feature_mask = selector.get_support()
selected_features = X_train_v2.columns[selected_feature_mask]
selected_features
```

get_support() returns a True/False mask. True means selected and False means not selected. Applying that mask to X_train_v2.columns returns the actual names of the selected features.

9. Plotting SelectKBest F1 Scores

```
fig, ax = plt.subplots()

x = np.arange(1, 14)
y = f1_score_list

ax.bar(x, y, width=0.2)
ax.set_xlabel('Number of features selected using mutual information')
ax.set_ylabel('F1-Score(weighted)')
ax.set_ylim(0, 1.2)
ax.set_xticks(np.arange(1, 14))
ax.set_xticklabels(np.arange(1, 14), fontsize=12)

for i, v in enumerate(y):
    plt.text(x=i+1, y=v+0.05,
             s=str(round(v, 3)), ha='center')

plt.tight_layout()
```

A previous graph issue came from using round(v), which rounds to zero decimal places. For example, 0.92 becomes 1. Using round(v, 3) keeps three decimal places.

10. Recursive Feature Elimination (RFE)

```
from sklearn.feature_selection import RFE
```

```

rfe_f1_score_list = []

for k in range(1, 14):
    RFE_selector = RFE(
        estimator=gbc,
        n_features_to_select=k,
        step=1
    )
    RFE_selector.fit(X_train_v3, y_train_v3)

    sel_X_train_v3 = RFE_selector.transform(X_train_v3)
    sel_X_test_v3 = RFE_selector.transform(X_test_v3)

    gbc.fit(sel_X_train_v3, y_train_v3)
    RFE_preds = gbc.predict(sel_X_test_v3)

    f1_score_rfe = round(
        f1_score(y_test_v3, RFE_preds, average='weighted'), 3
    )
    rfe_f1_score_list.append(f1_score_rfe)

```

Explanation: RFE uses the chosen model to rank features and recursively removes the least important ones. step=1 removes one feature at a time. The loop tests from 1 to 13 remaining features.

SelectKBest vs RFE: SelectKBest asks which features are individually informative about the target. RFE asks which features are useful according to the selected model. SelectKBest is generally simpler and faster; RFE is model-based and can take longer.

11. Boruta Feature Selection

```

from boruta import BorutaPy

boruta_selector = BorutaPy(gbc, random_state=42)

boruta_selector.fit(
    X_train_v4.values,
    y_train_v4.values.ravel()
)

sel_X_train_v4 = boruta_selector.transform(X_train_v4.values)
sel_X_test_v4 = boruta_selector.transform(X_test_v4.values)

gbc.fit(sel_X_train_v4, y_train_v4)
boruta_preds = gbc.predict(sel_X_test_v4)

boruta_f1_score = round(
    f1_score(y_test_v4, boruta_preds, average='weighted'), 3
)

```

Explanation: Boruta uses the estimator's feature importance and compares real features with randomized shadow features to identify genuinely relevant features. It is different from simply requesting a fixed number of top features.

Important correction: The F1-score must compare y_test_v4 with boruta_preds. Using y_train_v4 with predictions generated from the test set would compare different datasets.

Why .values was needed: In this setup Boruta's transform operation can expect NumPy arrays. Passing a Pandas DataFrame caused an indexing error, so X_train_v4.values and X_test_v4.values were used consistently.

12. Finding the Features Selected by Boruta

```

selected_features_mask = boruta_selector.support_

selected_features = X_train_v4.columns[selected_features_mask]

```

```
selected_features
```

Line 1: `boruta_selector.support_` gives Boruta's Boolean selection mask. True means Boruta selected the feature; False means it did not.

Line 2: `X_train_v4.columns` contains the feature names. Applying the Boolean mask keeps only the names corresponding to True values.

Line 3: Writing `selected_features` by itself displays the selected feature names in Jupyter or Colab.

In simple terms: **ask Boruta which features were selected → use the True/False mask to filter the column names → display the names.**

13. Final Feature-Selection Comparison Graph

```
fig, ax = plt.subplots()

x = [
    'All features (13)',
    'Variance threshold (11)',
    'Filter - MI (3)',
    'RFE(3)',
    'Boruta (9)'
]

y = [f1_score_all, f1_score_var, 0.981, 1.0, boruta_f1_score]

ax.bar(x, y, width=0.2)

ax.set_xlabel('Feature selection method')
ax.set_ylabel('F1-Score (weighted)')
ax.set_ylim(0, 1.2)

for i, v in enumerate(y):
    plt.text(
        x=i, y=v+0.01,
        s=str(round(v, 3)),
        ha='center'
    )

plt.tight_layout()
```

This graph compares five configurations: all 13 features, the 11-feature variance experiment, three features selected by Mutual Information, three features selected by RFE, and nine features selected by Boruta. The taller the bar, the higher the weighted F1-score.

The values 0.981 and 1.0 are manually entered. If they are the actual results for MI with 3 features and RFE with 3 features, the graph is fine. A more robust approach is to save those results in variables and use the variables in the graph.

14. Comparison of the Feature-Selection Methods

Method	Question	Main idea
Variance	Does the feature vary much?	Removes very low-variance features.
Mutual Information	How informative is the feature about the target?	Ranks individual features using an information-based score.
RFE	Which features help my model?	Recursively removes less important features.
Boruta	Is the feature genuinely relevant?	Compares real feature importance with shadow features.

15. The Big Picture

The practical is about determining whether the model can maintain or improve predictive performance while using fewer features. The baseline establishes a reference point. Each feature-selection method produces a different subset of the original features; the Gradient Boosting model is trained using that subset and evaluated using weighted F1-score. The final graph makes the results easy to compare.

The key pattern to remember is: **fit the selector on training data → transform training data → transform test data using the same selector → train the model → predict on test data → compare predictions with test labels.** This helps prevent test-set leakage.